

**(1) LAB EXERCISE: Write a simple "Hello World" program in two different programming languages of your choice. Compare the structure and syntax.**

**Ans:**

php-

```
<?php
    echo "Hello World";
?>
```

Java script-

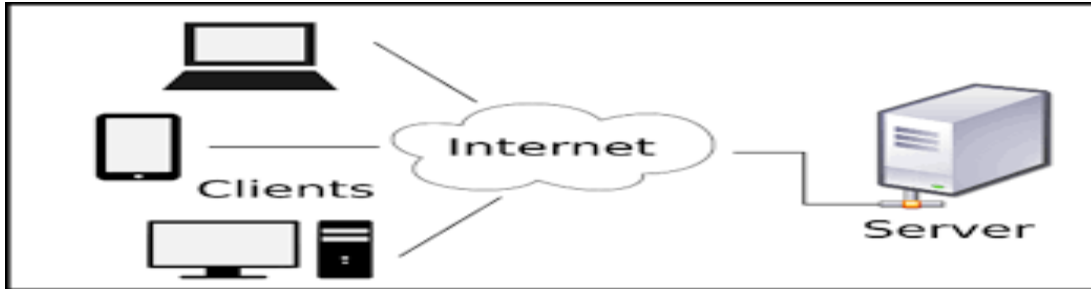
```
console.log("Hello World");
```

Comparison: Structure and Syntax

| Feature           | PHP                                         | JavaScript                                                    |
|-------------------|---------------------------------------------|---------------------------------------------------------------|
| Use Case          | Server-side scripting (runs on web server)  | Client-side scripting (runs in browser or server via Node.js) |
| Start/Entry Point | Starts with <?php . . . ?> tag              | Runs top-down (no special tag required)                       |
| Output Statement  | echo                                        | console.log()                                                 |
| Semicolon (;)     | Required after statements                   | Recommended, but often optional                               |
| Environment       | Requires a server like XAMPP, WAMP, or LAMP | Runs directly in browsers or via Node.js                      |
| File Extension    | .php                                        | .js                                                           |

**(2) Research and create a diagram of how data is transmitted from a client to a server over the internet.**

**Ans:-** Data transmission from a client to a server over the internet involves several steps, including a client request, server processing, and data return, all using network protocols like TCP/IP and HTTP.



**(3) Design a simple HTTP client-server communication in any language.**

**Ans:-**<?php

```
// client.php //
```

```
$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, "http://localhost/server.php");
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
$response = curl_exec($ch);
curl_close($ch);
$data = json_decode($response, true);
echo "Server Response: " . $data['message'];
?>
```

<?php

```
// server.php //
```

```
header('Content-Type: application/json');
$data = [
    "message" => "Hello from the server!",
    "status" => "success"
];
echo json_encode($data);
?>
```

Output - Server Response: Hello from the server!

**(4) Research different types of internet connections (e.g., broadband, fiber, satellite) and list their pros and cons.**

**Ans:- 1. DSL (Digital Subscriber Line)**

**Pros:**

- Widely available (uses existing telephone lines)
- Usually affordable
- Dedicated line (not shared with neighbors)

**Cons:**

- Slower than newer technologies
- Speed decreases with distance from provider's central office

**2. Cable Internet**

**Pros:**

- Faster than DSL in many areas
- Good for streaming and online gaming
- Widely available in urban and suburban areas

**Cons:**

- Shared bandwidth (speed may drop during peak hours)
- Higher cost than DSL

**3. Fiber Optic Internet**

**Pros:**

- Extremely fast speeds (up to 1 Gbps or more)
- Low latency — ideal for gaming, video conferencing
- More reliable than copper lines

#### Cons:

- Expensive installation
- Limited availability in rural areas

### 4. Satellite Internet

#### Pros:

- Available virtually anywhere (good for remote/rural areas)
- Doesn't require cable or phone lines

#### Cons:

- High latency (signal travels to space and back)
- Slower speeds and expensive
- Weather can affect signal quality

(5): Simulate HTTP and FTP requests using command line tools (e.g., curl).

Ans:-

#### 1. Simulate an HTTP Request Using

##### GET Request (Fetch a Web Page)

```
curl http://example.com
```

##### POST Request (Send Data to a Server)

```
curl -X POST -d "username=john&password=1234" http://example.com/login.php
```

Save Output to a File

```
curl -o page.html http://example.com
```

HTTPS with Header

```
curl -H "Authorization: Bearer YOUR_TOKEN" https://api.example.com/data
```

## 2. Simulate an FTP Request

Upload a File Using

```
curl -T file.txt ftp://ftp.example.com/ --user username:password
```

Uploads file.txt to the FTP server.

Download a File Using

```
curl -O ftp://ftp.example.com/file.txt --user username:password
```

Downloads file.txt from the server.

**(5): Identify and explain three common application security vulnerabilities. Suggest Possible solutions.**

**Ans:-**

### 1. SQL Injection

Explanation:

SQL Injection occurs when an application allows users to enter input directly into a SQL query without proper validation or escaping. This can allow attackers to execute arbitrary SQL commands, potentially accessing or destroying database data.

Example:

```
SELECT * FROM users WHERE username = '$username' AND password = '$password';
```

Solution:

- Use Prepared Statements (Parameterized Queries) instead of string concatenation.

Example in PHP (PDO):

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND password = ?");  
$stmt->execute([$username, $password]);
```

- - Validate and sanitize all user inputs.
  - Use ORM tools (e.g., Eloquent in Laravel) that abstract SQL queries securely.

## 2. Cross-Site Scripting (XSS)

Explanation:

XSS occurs when attackers inject malicious scripts into web pages that are viewed by other users. These scripts can steal cookies, session tokens, or manipulate DOM.

Types:

- Stored XSS: Malicious script stored on the server.
- Reflected XSS: Script comes from the current request (e.g., URL).
- DOM-based XSS: Manipulation of the DOM environment in the browser.

Solution:

- Escape output before rendering data into HTML using frameworks (e.g., {{ data }} in Blade or JSX).
- Use Content Security Policy (CSP) headers to restrict script execution.
- Validate and sanitize input using libraries (e.g., DOMPurify for HTML).
- Avoid innerHTML and use textContent or safer DOM methods.

## 3. Cross-Site Request Forgery (CSRF)

Explanation:

CSRF tricks an authenticated user into making a request that they did not intend (e.g., changing password, transferring money) by exploiting their session cookies.

Example:

A user is logged into a banking site. An attacker sends a malicious link:

```

```

Solution:

- Use CSRF Tokens:
  - Include a unique token in every state-changing request and verify it on the server.
  - Example in Laravel: @csrf directive in forms.
- Use SameSite cookies to restrict when cookies are sent.
- Require re-authentication for sensitive actions.

**(6)Identify and classify 5 applications you use daily as either system software or application software.**

**Ans:-** Here are 5 applications I use daily, classified as either system software or application software:

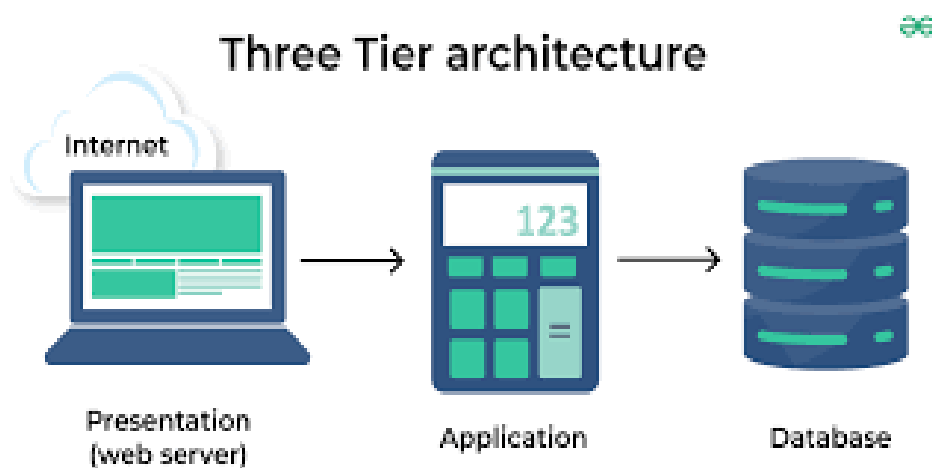
1. Google Chrome – Application Software  
(Used for browsing the internet)
2. Microsoft Word – Application Software  
(Used for creating and editing documents)
3. Windows Operating System – System Software  
(Manages computer hardware and provides a platform for applications)
4. File Explorer – System Software  
(Used to manage files and folders on the system)

5. Spotify – Application Software  
(Used for streaming music)

**(7) Design a basic three-tier software architecture diagram for a web application**

**Ans: -**

The Three-Tier Client-Server Architecture divides systems into presentation, application, and data layers, increasing scalability, maintainability, and efficiency. By separating the concerns, this model optimizes resource management and allows for independent scaling and updates, making it a popular choice for complex distributed systems.



**(8) Create a case study on the functionality of the presentation, business logic, and data access layers of a given software system.**

**Ans: -**

Case Study: Student Management System (SMS)

Overview



The Student Management System (SMS) is a web-based application used by administrators, teachers, and students to manage student records, enrollments, grades, and attendance.

It follows a three-tier architecture:

1. Presentation Layer
2. Business Logic Layer
3. Data Access Layer

## 1. Presentation Layer (User Interface Layer)

Function:

- Provides the interface for users to interact with the system.
- Sends user inputs to the business logic layer.
- Displays results or errors returned from the logic or data layer.

Example in SMS:

- A student logs in to check their grades.
- A teacher fills out a form to update attendance.
- An admin adds a new course.

Technologies:

- HTML, CSS, JavaScript
- Front-end frameworks: React, Angular, or Blade (Laravel)

## 2. Business Logic Layer (Application Layer)

Function:

- Processes input received from the presentation layer.

- Implements rules and workflows (e.g., GPA calculation, student promotion).
- Controls access and ensures data is valid before database interaction.

Example in SMS:

- Verifies student login credentials.
- Calculates a student's final grade.
- Validates whether a course can be assigned to a student based on prerequisites.

Technologies:

- PHP, Node.js, Python, Java (Laravel, Express.js, Django, Spring)

### 3. Data Access Layer (Database Layer)\

Function:

- Handles communication with the database.
- Executes CRUD (Create, Read, Update, Delete) operations.
- Ensures secure and optimized data access.

Example in SMS:

- Retrieves a list of students enrolled in a course.
- Saves attendance records submitted by a teacher.
- Updates a student's grade after an exam.

Technologies:

- MySQL, PostgreSQL, MongoDB
- ORM Tools: Eloquent (Laravel), Sequelize (Node.js), Hibernate (Java)

### How the Layers Work Together (Use Case: Grade Submission)

1. Teacher enters grades via the web interface (Presentation Layer).
2. The Application Layer validates the grades, checks course rules, and confirms the teacher's permission.
3. The Data Access Layer stores the grades in the student\_grades table in the database.
4. A confirmation message is returned up the layers and shown to the teacher.

### Benefits of Using Three-Tier Architecture

- Separation of concerns (UI, logic, and data are modular)
- Easier maintenance and updates
- Improved scalability and security
- Better testing and debugging

**(9)Explore different types of software environments (development, testing, production).Set up a basic environment in a virtual machine.**

**Ans:-**

#### 1. Exploring Different Types of Software Environments

Software environments are stages in the software development lifecycle that help ensure the application works as expected before going live.

##### a. Development Environment

- Purpose: Where developers write and test code.
- Characteristics:

- Frequently changes.
  - Has debugging tools.
  - Usually contains mock data.
- Example Tools:
  - IDEs (VS Code, PhpStorm)
  - Local servers (XAMPP, WAMP, Node.js)
  - Git for version control

## b. Testing Environment

- Purpose: Used by QA engineers or automated tools to test the application.
- Characteristics:
  - Simulates production but may use fake data.
  - Automated/manual tests are performed here.
  - Bugs and issues are identified.
- Tools:
  - Selenium, Postman, JUnit, PHPUnit
  - CI/CD Tools (GitHub Actions, Jenkins)

## c. Production Environment

- Purpose: The live environment where real users interact with the software.
- Characteristics:
  - Stable, secure, and optimized for performance.
  - Real-time data and logging.
  - Strict access control and monitoring.

## 2. Setting Up a Basic Environment in a Virtual Machine

## Tools Needed:

- A virtual machine tool like VirtualBox or VMware
- An ISO image of an OS (e.g., Ubuntu)
- Software stack (e.g., LAMP, Node.js)

## Step-by-Step Setup Example (Ubuntu + LAMP Stack)

### Step 1: Install VirtualBox & Create a VM

- Download and install VirtualBox
- Create a new VM:
  - Name: DevEnv
  - Type: Linux
  - Version: Ubuntu (64-bit)
  - RAM: At least 2048 MB
  - Create a virtual hard disk (e.g., 20 GB)

### Step 2: Install Ubuntu

- Download Ubuntu ISO from ubuntu.com
- Start the VM and select the ISO to install Ubuntu.

### Step 3: Install the LAMP Stack (Linux, Apache, MySQL, PHP)

```
sudo apt update
```

```
sudo apt install apache2
```

```
sudo apt install mysql-server
```

```
sudo apt install php libapache2-mod-php php-mysql
```

### Step 4: Test the Setup

- Place a test file in /var/www/html:

```
echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/info.php
```
- Open browser in VM: <http://localhost/info.php>

### Step 5: (Optional) Add Git and VS Code

```
sudo apt install git
```

```
sudo snap install code --classic
```

**(10) Create a list of software you use regularly and classify them into the following categories: system, application, and utility software.**

**Ans:- 1. System Software**

Software that manages hardware and runs the system.

| Software       | Purpose                                 |
|----------------|-----------------------------------------|
| Windows 10/11  | Operating system to manage the computer |
| Linux (Ubuntu) | Open-source operating system            |
| macOS          | Apple's desktop operating system        |
| Android OS     | Mobile operating system                 |

## 2. Application Software

Software used to perform specific user tasks.

| Software | Purpose |
|----------|---------|
|----------|---------|

|                |                                         |
|----------------|-----------------------------------------|
| Google Chrome  | Web browsing                            |
| Microsoft Word | Document creation and editing           |
| WhatsApp       | Messaging and voice/video communication |
| Zoom           | Online video meetings                   |
| Spotify        | Music streaming                         |
| VS Code        | Code editing and development            |
| Canva          | Graphic design                          |

### 3. Utility Software

Software that helps manage, maintain, and optimize a computer.

| Software         | Purpose                          |
|------------------|----------------------------------|
| Windows Defender | Antivirus and malware protection |
| WinRAR / 7-Zip   | File compression and extraction  |

|                                 |                                           |
|---------------------------------|-------------------------------------------|
| CCleaner                        | Disk cleanup and performance optimization |
| Backup Software                 | Creating backups of important data        |
| Disk Management Tool (built-in) | Disk partitioning and formatting          |

## **(11) Write a report on the various types of application software and how they improve productivity**

### **Ans:- 1. Introduction**

Application software refers to programs designed to perform specific tasks for users. These tools help individuals and organizations automate processes, communicate efficiently, and manage information, thereby improving productivity.

## **2. Types of Application Software**

### **a) Word Processing Software**

- Examples: Microsoft Word, Google Docs
- Purpose: Create, edit, format, and print documents.
- Productivity Benefits:
  - Streamlines document creation with templates and spell checkers.
  - Supports collaboration through real-time editing.
  - Enhances professionalism with formatting tools.

### **b) Spreadsheet Software**

- Examples: Microsoft Excel, Google Sheets
- Purpose: Organize, calculate, analyze, and visualize data in tabular form.



- Productivity Benefits:
  - Automates complex calculations using formulas and functions.
  - Provides tools for data visualization with charts and graphs.
  - Enables data sorting, filtering, and what-if analysis for better decision-making.

### c) Database Management Software

- Examples: MySQL, Oracle, Microsoft Access
- Purpose: Store, manage, and retrieve large volumes of structured data.
- Productivity Benefits:
  - Ensures quick access to accurate data.
  - Supports data integrity and security.
  - Facilitates reporting and data-driven decision-making.

### d) Communication Software

- Examples: Email clients (Outlook, Gmail), Zoom, Slack
- Purpose: Facilitate communication through email, video conferencing, and instant messaging.
- Productivity Benefits:
  - Enables real-time collaboration regardless of location.
  - Reduces delays through instant messaging and virtual meetings.
  - Supports file sharing and project coordination.

### e) Project Management Software

- Examples: Trello, Asana, Microsoft Project
- Purpose: Plan, track, and manage projects and tasks.
- Productivity Benefits:
  - Enhances task delegation and deadline management.
  - Improves team collaboration and transparency.
  - Provides dashboards and reports to monitor progress.

## f) Multimedia Software

- Examples: Adobe Photoshop, VLC Media Player
- Purpose: Create, edit, and play audio, video, and images.
- Productivity Benefits:
  - Enables professional content creation and editing.
  - Supports presentations and marketing materials.
  - Facilitates training and communication with multimedia resources.

## 3. Conclusion

Application software plays a critical role in enhancing productivity across various fields by automating tasks, improving communication, and enabling efficient data management. Choosing the right application software tailored to specific needs can significantly optimize workflows and business outcomes.

**(12) Create a flowchart representing the Software Development Life Cycle (SDLC).**

**Ans: -**



**(13) Write a requirement specification for a simple library management system**

**Ans:-**

Project: Library Management System

## 1. Introduction

### 1.1 Purpose

The purpose of this document is to specify the functional and non-functional requirements for a simple Library Management System (LMS) that automates the daily library operations such as managing books, users, and transactions.

### 1.2 Scope

The LMS is designed for small libraries or educational institutions. It will support basic features like book issuance, returns, member management, and inventory tracking.

### 1.3 Intended Users

- Librarian
- Students (Members)
- System Administrator

## 2. Overall Description

### 2.1 System Functions

- Manage books (add, update, delete, search)
- Manage members
- Issue and return books
- Track due dates and fines
- Generate reports

### 2.2 Assumptions and Dependencies

- The system will run on a web browser.
- The database is MySQL.
- Internet access may be required for updates or cloud backup.

## 3. Functional Requirements

### 3.1 User Authentication

- Users must log in to access features.
- Admins and librarians have different roles.

### 3.2 Book Management

- Admin can add, update, delete, or search for books.
- Each book has attributes: Title, Author, ISBN, Category, Availability.

### 3.3 Member Management

- Register new members (students).
- View member details and borrowing history.

### 3.4 Book Issuance and Return

- Issue books to members with issue and due date.
- Return books and calculate fines if overdue.
- Update book availability on issue/return.

### 3.5 Reporting

- Generate reports of issued books, overdue books, and fines collected.
- Report of books added per month.

## 4. Non-Functional Requirements

### 4.1 Performance

- System should handle up to 100 concurrent users without noticeable delay.

### 4.2 Usability

- User-friendly interface with clear navigation.
- Search functionality for books and members.

### 4.3 Security

- Only authenticated users can access or update records.
- Sensitive data (passwords) must be encrypted.

#### 4.4 Maintainability

- Code should follow a modular structure.\
- System should be easily upgradable.

### 5. System Requirements

#### 5.1 Hardware

- Server: 4 GB RAM, 100 GB storage
- Client: Any system with modern web browser

#### 5.2 Software

- Backend: PHP (Laravel) or Node.js
- Frontend: HTML, CSS, JavaScript
- Database: MySQL or PostgreSQL

### **(14)Perform a functional analysis for an online shopping system.**

**Ans:-** 1. Introduction

The Online Shopping System is a web-based application that allows users to browse, search, and purchase products over the internet. It also enables sellers/admins to manage product listings and track orders.

#### 2. Stakeholders

- Customer – Buys products
- Admin – Manages inventory and users
- Delivery Personnel – Delivers orders
- Payment Gateway – Processes online payment

#### 3. Core Functional Requirements

##### ♦ 3.1 User Registration and Login

- Customers can register, log in, and manage profiles.
- Admins log in with elevated privileges.

### ♦ 3.2 Product Browsing and Search

- Products are categorized (e.g., electronics, clothing).
- Users can filter/search products by price, rating, brand, etc.

### ♦ 3.3 Product Details View

- View product image, description, price, reviews, and availability.

### ♦ 3.4 Shopping Cart

- Add/remove products to cart.
- View total price and item summary.
- Update quantity before checkout.

### ♦ 3.5 Checkout and Payment

- Enter shipping information.
- Choose payment method (credit/debit card, wallet, COD).
- Integrate with secure payment gateways.

### ♦ 3.6 Order Management

- Users can view order history and status (pending, shipped, delivered).
- Admin can update order status.

### ♦ 3.7 Inventory Management (Admin)

- Add, update, or remove product listings.  
Manage stock levels and view sales reports.

### ♦ 3.8 Reviews and Ratings

- Users can leave reviews for purchased products.
- Reviews visible on product pages.

### ♦ 3.9 Notifications

- Send order confirmations, shipping updates, and promotional emails.

### 4. Optional/Advanced Features

- Wishlist for saving favorite products.
- Discount codes and promotions.
- Real-time chat support.
- Mobile app integration.

### 5. Non-Functional Requirements

- Security: Use HTTPS, encrypt user data, secure payment.
- Scalability: Handle high traffic during sales.
- Performance: Quick response time (<2 seconds for loading pages).
- Availability: 24/7 uptime with minimal downtime.
- Responsiveness: Mobile-friendly interface.

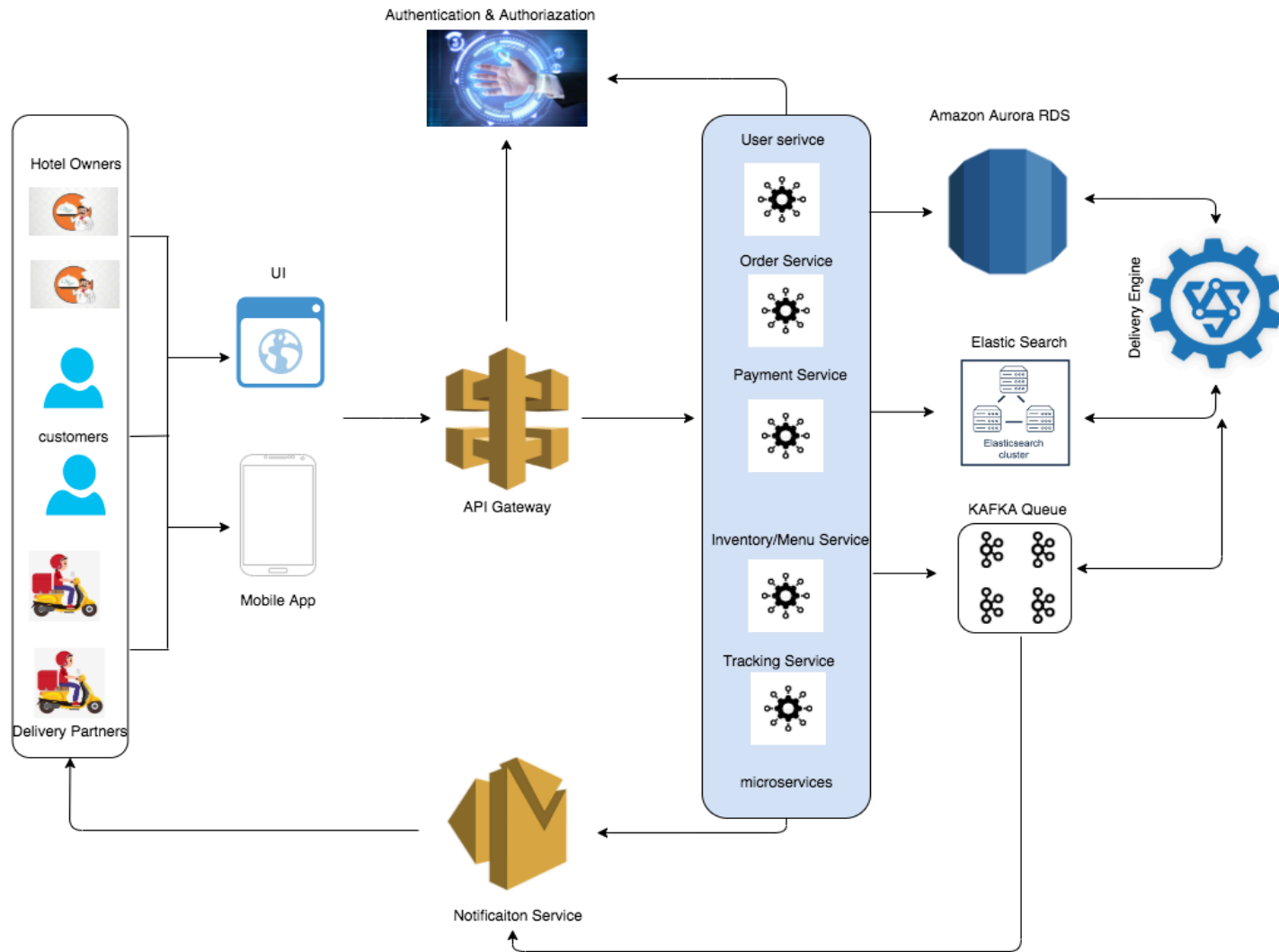
### 6. Functional Flow Example

Use Case: Purchase a Product

1. User logs in.
2. Browses or searches for a product.
3. Views product details and adds to cart.
4. Proceeds to checkout and completes payment.
5. Receives order confirmation.
6. Admin processes and updates order status.
7. Delivery team delivers the product.
8. User can rate and review the product.

**(15)Design a basic system architecture for a food delivery app**

**Ans:-**



(16) Develop test cases for a simple calculator program

Ans: -

| Test Case ID | Test Description                   | Input | Expected Output | Status |
|--------------|------------------------------------|-------|-----------------|--------|
| TC01         | Test addition of two positive ints | 5 + 3 | 8               | Pass   |



|      |                                      |                  |                          |      |
|------|--------------------------------------|------------------|--------------------------|------|
| TC02 | Test addition of negative numbers    | $-4 + (-6)$      | $-10$                    | Pass |
| TC03 | Test subtraction with zero           | $9 - 0$          | $9$                      | Pass |
| TC04 | Test subtraction resulting negative  | $3 - 7$          | $-4$                     | Pass |
| TC05 | Test multiplication of positives     | $4 \times 6$     | $24$                     | Pass |
| TC06 | Test multiplication with zero        | $9 \times 0$     | $0$                      | Pass |
| TC07 | Test division with no remainder      | $8 \div 2$       | $4$                      | Pass |
| TC08 | Test division with remainder         | $7 \div 2$       | $3.5$                    | Pass |
| TC09 | Test division by zero                | $5 \div 0$       | Error / Exception        | Pass |
| TC10 | Test with decimal inputs             | $2.5 + 3.2$      | $5.7$                    | Pass |
| TC11 | Test chained operations (if allowed) | $5 + 2 \times 3$ | $11$ (if follows BODMAS) | Pass |
| TC12 | Test invalid input (e.g. string)     | "abc" + 5        | Error / Invalid input    | Pass |

**(17) Document a real-world case where a software application required critical maintenance.**

**Ans:-**

Case Study: Critical Maintenance of Facebook's Outage – October 4, 2021

### Overview

On October 4, 2021, Facebook and its associated services – including Instagram and WhatsApp – experienced a global outage that lasted for approximately 6 hours. This case highlights the importance of software maintenance, configuration management, and system recovery.

### Background

Facebook performs routine maintenance and updates to improve performance, security, and scalability. On this occasion, a routine update to their backbone routers that coordinate network traffic between data centers went wrong.

### Issue Summary

- A configuration change in the Border Gateway Protocol (BGP) disrupted Facebook's DNS servers, making its services unreachable.
- Internal tools and systems that Facebook uses to diagnose and resolve problems were also impacted.
- Engineers were physically locked out of their data centers due to network-dependent access controls.

### Root Cause

A faulty BGP configuration update caused Facebook to effectively disappear from the internet by withdrawing its DNS routes.

## Critical Maintenance Actions Taken

### 1. Emergency Rollback

- Network engineers had to manually revert the BGP configurations.

### 2. Physical Access Required

- Technicians were sent to data centers to physically restore access to routers.

### 3. DNS Restoration

- DNS servers had to be re-advertised to the global internet infrastructure.

### 4. Gradual Service Restart

- Services were restarted in phases to avoid overwhelming the infrastructure.

## Outcome

- Services were restored after 6+ hours.
- Facebook issued a public apology and conducted a post-mortem analysis.

## Key Learnings

- Even minor configuration errors can lead to global service disruption.
- Critical systems must have redundant access paths for emergency maintenance.
- Monitoring and rollback mechanisms are essential for real-time diagnostics.
- Teams need protocols for physical intervention in case of remote failure.

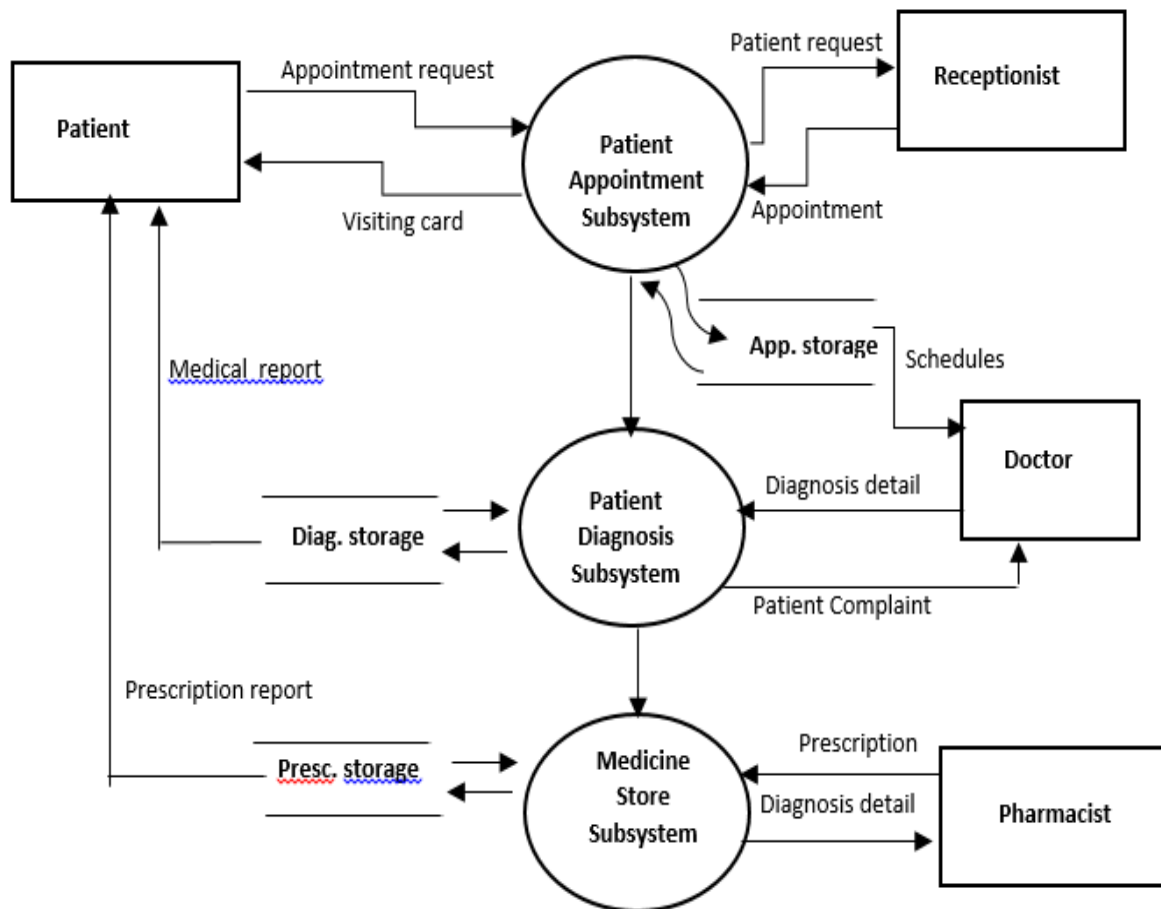
## Conclusion

This incident serves as a prime example of the need for critical software maintenance, robust configuration management, and disaster recovery planning.

Companies must regularly audit their systems to prepare for such high-stakes failures.

**(18)Create a DFD for a hospital management system.**

**Ans:-**



**(19)Build a simple desktop calculator application using a GUI library.**

**Ans:-**

<!DOCTYPE html>

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Calculator</title>
    <style>
      body {
        display: flex;
        justify-content: center;
        align-items: center;
        height: 100vh;
        margin: 0;
        background-color: #f0f0f0;
        font-family: Arial, sans-serif;
      }
      .calculator {
        background-color: #333;
        border-radius: 10px;
        padding: 20px;
        box-shadow: 0 0 10px rgba(0, 0, 0, 0.3);
      }
      .display {
        background-color: #fff;
        padding: 10px;
        margin-bottom: 10px;
        text-align: right;
        border-radius: 5px;
        font-size: 24px;
```

```
}  
.buttons {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 5px;  
}  
button {  
  padding: 15px;  
  font-size: 18px;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
  background-color: #666;  
  color: white;  
}  
button:hover {  
  background-color: #777;  
}  
.operator {  
  background-color: #ff9500;  
}  
.equals {  
  background-color: #2196f3;  
}  
.clear {  
  background-color: #f44336;  
}  
</style>  
</head>
```

```

<body>
  <div class="calculator">
    <div class="display" id="display">0</div>
    <div class="buttons">
      <button class="clear"
onclick="clearDisplay()">C</button>
      <button onclick="appendNumber('7')">7</button>
      <button onclick="appendNumber('8')">8</button>
      <button onclick="appendNumber('9')">9</button>
      <button class="operator"
onclick="setOperator('+')">+</button>
      <button onclick="appendNumber('4')">4</button>
      <button onclick="appendNumber('5')">5</button>
      <button onclick="appendNumber('6')">6</button>
      <button class="operator"
onclick="setOperator('-')">-</button>
      <button onclick="appendNumber('1')">1</button>
      <button onclick="appendNumber('2')">2</button>
      <button onclick="appendNumber('3')">3</button>
      <button class="operator"
onclick="setOperator('*')">*</button>
      <button onclick="appendNumber('0')">0</button>
      <button onclick="appendNumber('.')">.</button>
      <button class="equals" onclick="calculate()">=</button>
      <button class="operator"
onclick="setOperator('/')">÷</button>
    </div>
  </div>
  <script>
    let currentNumber = "0";

```

```
let previousNumber = "";
let operator = "";
let newNumber = true;

function updateDisplay() {
    document.getElementById("display").textContent =
currentNumber;
}

function appendNumber(num) {
    if (newNumber) {
        currentNumber = num;
        newNumber = false;
    } else {
        if (num === "." && currentNumber.includes("."))
return;
        currentNumber += num;
    }
    updateDisplay();
}

function setOperator(op) {
    if (previousNumber && !newNumber) {
        calculate();
    }
    operator = op;
    previousNumber = currentNumber;
    newNumber = true;
}
```



```
function calculate() {
  if (!previousNumber || newNumber) return;
  let result;
  const prev = parseFloat(previousNumber);
  const current = parseFloat(currentNumber);
  switch (operator) {
    case "+":
      result = prev + current;
      break;
    case "-":
      result = prev - current;
      break;
    case "*":
      result = prev * current;
      break;
    case "/":
      result = prev / current;
      break;
    default:
      return;
  }
  currentNumber = result.toString();
  operator = "";
  previousNumber = "";
  newNumber = true;
  updateDisplay();
}
```

```

function clearDisplay() {
    currentNumber = "0";
    previousNumber = "";
    operator = "";
    newNumber = true;
    updateDisplay();
}
</script>
</body>
</html>

```

**(20) Draw a flowchart representing the logic of a basic online registration system**

**Ans: -**

Student registration system:-

